

Protected Terms Masking

What it is

Later in this pipeline, we run a **lemmatizer** — a tool that reduces words to their base ("dictionary") form so search works better. For example, it turns "manufacturing," "manufactures," and "manufactured" all into "manufacture," so a search for one of those words also finds documents using the others.

That's great for ordinary words. It's a disaster for brand names, product codes, and certificate codes, because a lemmatizer doesn't know the difference between an ordinary English word and a proper noun that happens to look like one. If it saw the brand name "Bunting" (a real magnet brand in this project's data), it could reduce it to "bunt" — turning a company's brand identity into a nonsense verb stem. The word is gone, and so is any hope of matching it in search.

Protected Terms Masking solves this by temporarily hiding those special terms before lemmatization runs, and putting the exact original text back afterward. "Masking" here just means: find the term, swap it for a meaningless placeholder token, remember what the placeholder stood for, and continue processing. "Unmasking" is the reverse: find the placeholder, look up what it stood for, and put the real text back.

Real-life analogy

Imagine a photo lab that touches up every photo run through it — sharpening, color-correcting, cropping. It's an automated batch process; it doesn't look closely at each photo, it just applies the same adjustments to everything that comes through.

Now imagine one of the photos has a small note taped to the corner with someone's exact, hand-written signature on it — something that must never be smoothed, sharpened, or "corrected" in any way, because any alteration destroys its value as a signature.

Before running the photo through the touch-up machine, you'd cover that one note with an opaque sticky label. The machine processes the whole photo as normal — it doesn't know or care that something is hidden under the label. Once the photo comes out the other side, you peel the label off and the signature is exactly as it was: untouched, because it was never actually exposed to the machine.

The brand names and codes ("Piranha," "ABS," "ISO 9001") are the signature. The lemmatizer is the touch-up machine. The placeholder token is the sticky label — deliberately blank and meaningless so the machine has nothing to "improve."

Why we used it here

This directly implements **R8** (Protected Terms) and its acceptance criterion **AC-2**, which is treated as a hard, non-negotiable correctness rule in this project (see `.claude/rules/bilingual-nlp.md`): masking must happen **before** lemmatization, and unmasking must happen **after**. If you lemmatize first and try to "protect" the term afterward, it's too late — the brand name has already been mangled, and there's no way to know what it used to say.

Real examples from this project's actual dataset (`bilingual_etl/nlp/whitelists/brands_whitelist.json`, 2,503 protected terms) make the risk concrete:

- **"Piranha"** — a pump brand from Zultzer Pumpen Kft. A generic lemmatizer has no reason to know this is a brand name rather than the fish; it could stem it into something unrecognizable.
- **"ABS"** — a pump brand. Note the whitelist also contains **"ABS MF"** and **"ABS XFP"** — longer terms that start with "ABS". This is exactly the scenario the longest-first sorting (see Code walkthrough) exists to handle correctly.
- **"ISO 9001"** — a certification code. Lemmatizers and tokenizers can behave unpredictably around numbers and mixed alphanumeric strings; masking sidesteps the question entirely by removing it from the lemmatizer's view.

The masking/unmasking logic lives in `bilingual_etl/nlp/protected_terms.py`. It sits in the pipeline right after HTML cleaning (`bilingual_etl/transform/html_cleaner.py`) and right before lemmatization — see Code walkthrough below for both.

Code walkthrough

Step 0 (earlier in the pipeline): clean the HTML first

Before protected-terms masking ever sees the text, `html_cleaner.py` has already stripped out HTML tags like `<p>`, `<strong>`, and `<br>` from the raw category descriptions. This matters because masking (and everything after it) works on plain prose — if `<strong>Bunting</strong>` were passed in with tags still attached, the regex matching in the masker could behave inconsistently around the tag boundaries.

```
BLOCK_TAGS = {"p", "br", "div", "hr", "li"}

def strip_html(text: str) -> str:
    if not text:
        return ""

    soup = BeautifulSoup(text, "html.parser")

    for tag in soup.find_all(BLOCK_TAGS):
        tag.insert_before(" ") # add a space where the tag was, so
        tag.insert_after(" ") # "word</p><p>word" doesn't glue into "wordword"

    plain = soup.get_text() # discard all tags, keep only the text
    return " ".join(plain.split()) # collapse any run of whitespace into single spaces
```

By the time protected-terms masking runs, the text is already clean prose — no tags, no double spaces, nothing to trip up the regex.

The whitelist and the module docstring

```
"""
Protected Terms Masking (R8)

Brand names, product codes, and certificate codes (e.g. "Bunting", "ISO 9001")
must survive lemmatization unchanged. Lemmatizers reduce words to their base
form ("Bunting" -> "bunt"), which destroys brand names.
```

```
The fix: mask every protected term with a placeholder BEFORE lemmatization,
then unmask (restore the original text) AFTER. This ordering is a correctness
requirement (AC-2), not a style choice – reversing it corrupts protected terms.
"""
```

```
DEFAULT_WHITELIST_PATH = Path(__file__).parent / "whitelists" / "brands_whitelist.json"
PLACEHOLDER_TEMPLATE = "zzzptermzzz{index}zzz"
PLACEHOLDER_PATTERN = re.compile(r"zzzptermzzz(\d+)zzz", re.IGNORECASE)
```

- `DEFAULT_WHITELIST_PATH` points at a JSON file with 2,503 entries — every brand, certificate code, and company name pulled from this project's actual dataset (including "Piranha," "ABS," "ABS MF," "ABS XFP," and "ISO 9001").
- `PLACEHOLDER_TEMPLATE` is the format for a placeholder token: `zzzptermzzz0zzz`, `zzzptermzzz1zzz`, and so on. It's deliberately an **all-lowercase, single unbroken run of letters and digits, with no spaces or punctuation**. This matters: a lemmatizer or tokenizer works by splitting text on word boundaries and punctuation. If the placeholder looked like `PROTECTED-TERM-0` or `Protected Term #0`, the tokenizer might split it into multiple pieces or try to "lemmatize" a piece of it. A single indivisible nonsense word like `zzzptermzzz0zzz` looks like one unknown word, which every tokenizer leaves alone.
- `PLACEHOLDER_PATTERN` is the regex used later to *find* placeholders during unmasking, with `re.IGNORECASE` — because if the lemmatizer happens to lowercase everything (a common lemmatizer behavior), the placeholder is already lowercase, but this guards against any case variation anyway.

Building one big search pattern from the whitelist

```
def _build_pattern(terms: list[str]) -> re.Pattern:
    # Longest terms first so "ABS MF" matches before "ABS" at the same position.
    sorted_terms = sorted(terms, key=len, reverse=True)
    alternation = "|".join(re.escape(t) for t in sorted_terms)
    return re.compile(rf"(?<!\w)(?:{alternation})(?!\w)", re.IGNORECASE)
```

This builds a single regex that can match any of the 2,503 protected terms in one pass. Three ideas worth unpacking in plain language:

- **Alternation (`|`)** — a regex option meaning "match A, or B, or C, ...". A regex engine tries the alternatives **left to right** and stops at the *first one that matches*, starting from the current position in the text. That's why `sorted_terms` puts the longest terms first: if "ABS" (3 characters) were listed before "ABS MF" (6 characters), the engine would find "ABS" inside "ABS MF" first, declare victory, and never even consider whether the longer, more specific term also matched. Sorting longest-first means the engine always gets a chance to try "ABS MF" before it settles for the shorter "ABS."
- **Word-boundary lookarounds** — `(?<!\w)` and `(?!\w)` — these say "don't match here if there's a word character immediately before/after." Think of them as a bouncer standing on either side of the match, checking that whatever it lets through isn't just a fragment of a bigger word. Without this, the pattern would happily match "ABS" inside the middle of the English word "absolute," which would silently and incorrectly mask a completely unrelated word. `(?<!\w)` is a **lookbehind** (checks what's just before the current position without consuming it), and `(?!\w)` is a **lookahead** (checks what's just after) — both are "look, but don't eat" checks that don't add extra characters to the match itself.
- `re.IGNORECASE` — the whole pattern matches regardless of letter case, so "Bunting," "bunting," and "BUNTING" are all caught by the same rule, even though the whitelist only stores one canonical spelling.
- `re.escape(t)` — protected terms can contain characters like periods or plus signs, which have special meaning in a regex. `re.escape` converts a term like "ISO 9001" into a literal-only pattern that matches those exact characters, not some regex trick.

Masking: hide the term, remember what it was

```
class ProtectedTermsMasker:
    def __init__(self, whitelist_path: Path = DEFAULT_WHITELIST_PATH):
        self._terms = load_whitelist(whitelist_path)
        self._pattern = _build_pattern(self._terms)

    def mask(self, text: str) -> tuple[str, dict[str, str]]:
        if not text:
            return text, {}

        mapping: dict[str, str] = {}
        counter = 0

        def _replace(match: re.Match) -> str:
            nonlocal counter
            placeholder = PLACEHOLDER_TEMPLATE.format(index=counter)
            mapping[placeholder] = match.group(0) # store the EXACT text as found, original
            counter += 1
            return placeholder

        masked_text = self._pattern.sub(_replace, text)
        return masked_text, mapping
```

- `self._pattern.sub(_replace, text)` scans text for every match of the big alternation pattern and calls `_replace` on each one.
- `match.group(0)` is the exact substring that was matched — not the whitelist's canonical spelling, but whatever casing actually appeared in the sentence (e.g. if the source text said "bunting" in lowercase, `match.group(0)` is "bunting", preserved as-is).
- Each match gets its own placeholder (`zzzptermzzz0zzz`, `zzzptermzzz1zzz`, ...) via the counter, and the mapping dict remembers placeholder -> original text. This mapping travels alongside the masked text through lemmatization and is needed later to unmask.

Unmasking: find the placeholder, restore the original

```
def unmask(self, text: str, mapping: dict[str, str]) -> str:
    if not mapping:
        return text

    def _restore(match: re.Match) -> str:
        placeholder = match.group(0).lower() # normalize case in case lemmatizer lowercased
        it
        return mapping.get(placeholder, match.group(0))

    return PLACEHOLDER_PATTERN.sub(_restore, text)
```

- After lemmatization has run on the masked text (safely, since it only ever saw meaningless placeholder tokens where the protected terms used to be), `unmask` scans for anything matching `PLACEHOLDER_PATTERN`.
- `match.group(0).lower()` lowercases whatever placeholder text was found, because the mapping's keys were built as exact-lowercase strings (`zzzptermzzz0zzz`) — this guards against a lemmatizer that lowercases the whole text as a side effect.
- `mapping.get(placeholder, match.group(0))` looks up the original text and substitutes it back in. The `match.group(0)` fallback means: if for some reason a placeholder-looking string appears that isn't in the mapping, leave it untouched rather than crashing or silently dropping text.

- The net effect: whatever the lemmatizer did to the rest of the sentence, the protected term comes back **exactly as it originally appeared** — same spelling, same capitalization, same everything.

How to explain this in an interview/to a teammate

"Our search pipeline lemmatizes text — reducing words to their base form — so that searches match related word forms. The problem is that brand names and codes like 'Piranha,' 'ABS,' and 'ISO 9001' aren't ordinary words, and lemmatizing them would corrupt them, for example turning 'Bunting' into 'bunt.' We fix this by masking: before lemmatization runs, we scan the text with a regex built from a whitelist of about 2,500 known terms, replace each match with a placeholder token, and remember what each placeholder stood for. Lemmatization then runs safely on text that no longer contains the fragile terms, and afterward we unmask — swap every placeholder back for its original text, exact casing included. The two tricky details are that we sort the whitelist longest-term-first so multi-word terms like 'ABS MF' aren't shadowed by a shorter match like 'ABS,' and we require word boundaries around each match so we never mask part of an unrelated word like 'absolute.'"